

Docker Registries & End-to-End Deployment Demo

In this demo, we will take a simple Spring Boot application backed by MySQL and move it through the complete Docker workflow:

```
Spring Boot Application
    ↓
Docker Image
    ↓
Docker Compose
    ↓
Spring Boot + MySQL Containers
    ↓
Docker Hub
    ↓
Ubuntu Server
```

The goal is to understand how Docker images are built, stored, distributed, and finally used to deploy an application on another machine.

1. Why Do Container Registries Exist?

A Docker image is a packaged application containing:

- Filesystem layers
- Application files
- Runtime
- Libraries and dependencies
- Metadata describing how the container should start

Suppose we build an image:

```
docker build -t spring-app .
```

Running:

```
docker images
```

might show:

REPOSITORY	TAG	IMAGE ID
spring-app	latest	a8b24c7...

At this point, the image exists only inside the **local Docker image store** of that machine.

That becomes a problem when an application has to move through multiple environments:

```
Developer
  ↓
Testing
  ↓
Staging
  ↓
Production
```

We do not want to manually copy the source code or JAR to every environment. Instead, the usual flow is:

```
Build once
  ↓
Store the deployable artifact centrally
  ↓
Every environment downloads the same artifact
```

That central system is a **Container Registry**.

| A container registry is an artifact distribution system for container images.

2. Registry vs Repository vs Image vs Tag

Consider this image reference:

```
docker.io/coderarmy/spring-app:1.2.0
```

It can be broken down as:

```
docker.io
  |
  └─ Registry

coderarmy
  |
  └─ Namespace / Account

spring-app
  |
  └─ Repository

1.2.0
  |
  └─ Tag
```

The general Docker image reference looks like:

```
[HOST[:PORT]]/NAMESPACE/REPOSITORY[:TAG]
```

If the registry host is omitted, Docker normally uses Docker Hub.

If the tag is omitted, Docker uses:

```
latest
```

Registry

A registry is the server or service that stores container images.

Think of it as a **warehouse**.

Repository

A repository contains related versions of the same application.

Example:

```
coderarmy/spring-app
```

Think of it as a **shelf inside the warehouse**.

Image

An image is an actual built application artifact.

For example:

```
spring-app version 1.2.0
```

Tag

A tag is a human-readable reference associated with an image.

Examples:

```
1.0.0  
1.2.0  
latest  
staging  
production
```

3. Public and Private Repositories

A container repository can broadly be public or private.

Public Repository

Anyone can pull the image.

Example:

```
docker pull coderarmy/spring-app:1.0.0
```

Public repositories are commonly used for:

- Open-source software
- Public libraries
- Tutorials
- Base images

Examples of commonly available public images include:

```
nginx  
mysql  
redis  
ubuntu
```

Private Repository

Only authorized users or services can access the image.

Private repositories are commonly used for:

- Backend services
- Internal microservices
- Payment applications
- Enterprise software
- Proprietary applications

Private means **access-controlled**. It does not automatically mean that the image itself is secure.

4. Docker Hub

Docker Hub is a container registry used to store, manage, and distribute Docker images.

A useful comparison is:

```
GitHub      → Source Code
Docker Hub  → Built Docker Images
```

Suppose the Docker Hub username is:

```
codarmy
```

A repository might be:

```
codarmy/spring-app
```

Different application versions can then be stored as:

```
codarmy/spring-app:1.0.0
codarmy/spring-app:1.1.0
codarmy/spring-app:2.0.0
```

5. AWS ECR

If an application's infrastructure runs primarily on AWS services such as:

```
EC2
ECS
EKS
Lambda Containers
```

Docker Hub can still be used.

However, AWS also provides its own managed container registry:

Amazon Elastic Container Registry — ECR

It performs the same fundamental job of storing and distributing container images while integrating with the AWS ecosystem.

6. Google Artifact Registry

Google Cloud provides **Artifact Registry** for storing container images and other artifacts.

An image reference might look like:

```
asia-south1-docker.pkg.dev/my-project/backend/spring-app:1.0.0
```

Breakdown:

```
asia-south1-docker.pkg.dev
    ↓
Registry Host

my-project
    ↓
GCP Project

backend
    ↓
Artifact Registry Repository

spring-app
    ↓
Image

1.0.0
    ↓
Tag
```

Google's older Container Registry was deprecated, and Google recommends using **Artifact Registry**.

7. Harbor

Docker Hub, ECR, and Google Artifact Registry are managed services.

Some organizations may instead want to operate their own container registry infrastructure.

One popular option is **Harbor**.

Harbor is a self-managed OCI registry commonly used when organizations need more control over their container infrastructure.

It provides features such as:

- Projects
 - Users
 - Authentication
 - Access control
 - Replication
 - Auditing
 - Vulnerability scanning
-

8. End-to-End Docker Demo

Our complete workflow will be:

```
Developer writes Spring Boot application
      ↓
Tests application locally
      ↓
Code gets merged
      ↓
Build Docker image
      ↓
Run Spring Boot + MySQL using Docker Compose
      ↓
Test everything locally
      ↓
Tag application image
      ↓
```

Push application image to Docker Hub



Ubuntu server pulls image



Ubuntu runs application + MySQL

9. Application Used in the Demo

We will use a deliberately small Spring Boot application.

API:

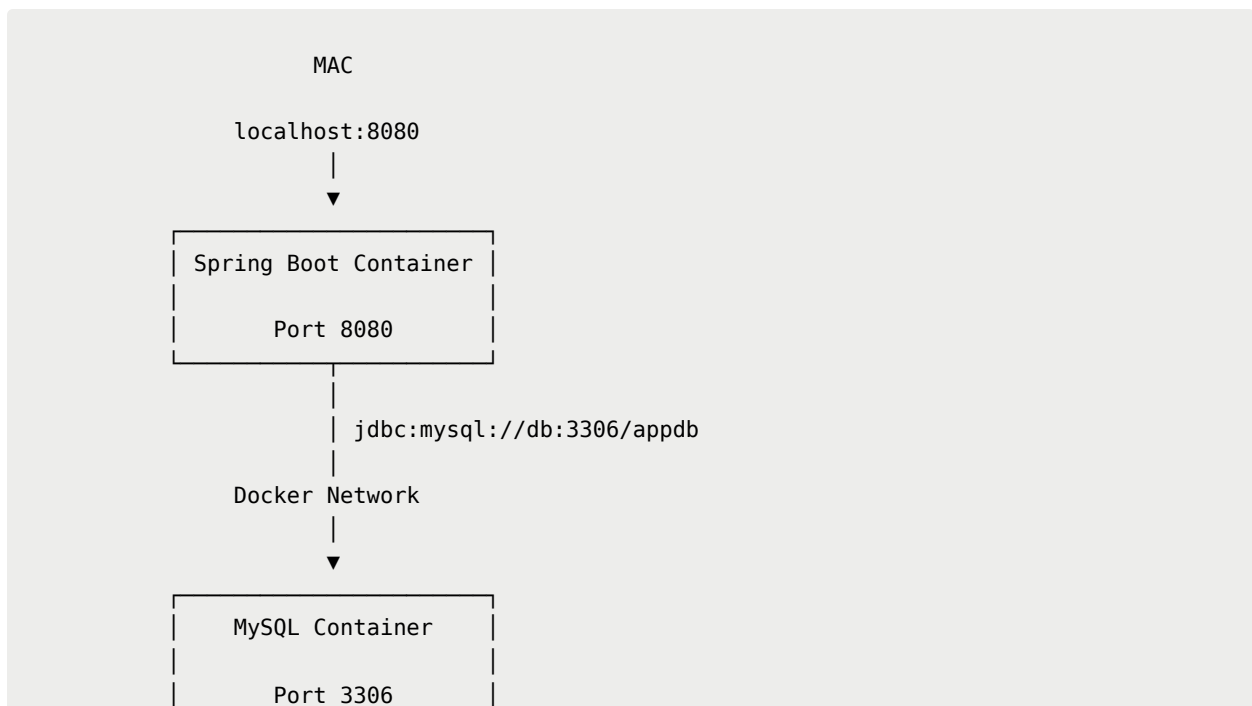
```
POST /notes?text=Hello
```

The application stores:

```
id | text
```

inside MySQL.

The local Docker architecture will look like:





From a networking perspective, the Spring Boot and MySQL containers are separate environments.

They communicate using Docker networking.

10. Prerequisites

For this demo:

- Java 21
- Docker Desktop
- IntelliJ IDEA or another IDE
- Terminal
- Multipass

Check Docker:

```
docker --version
```

Check Docker Compose:

```
docker compose version
```

Test Docker:

```
docker run hello-world
```

Docker Desktop already includes Docker Compose.

11. Project Location

For the local demo, the project is stored at:

```
~/Desktop/spring-mysql-demo
```

12. Multi-Stage Dockerfile

Create a file named:

```
Dockerfile
```

next to:

```
pom.xml
```

Use:

```
# =====  
# Stage 1: Build  
# =====  
  
FROM maven:3.9.13-eclipse-temurin-21 AS build  
  
WORKDIR /app  
  
COPY pom.xml .  
  
RUN mvn dependency:go-offline -B  
  
COPY src ./src  
  
RUN mvn clean package -DskipTests  
  
# =====  
# Stage 2: Production  
# =====
```

```
FROM eclipse-temurin:21-jre-alpine AS production

WORKDIR /app

COPY --from=build \
    /app/target/demo-0.0.1-SNAPSHOT.jar \
    app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

13. Understanding the Multi-Stage Build

The first stage is responsible for compiling the application.

```
FROM maven:3.9.13-eclipse-temurin-21 AS build
```

This image provides everything required for building the project, including:

```
Linux
Java JDK
Maven
```

The source code is compiled and packaged into a JAR.

The second stage starts with:

```
FROM eclipse-temurin:21-jre-alpine AS production
```

Instead of carrying Maven and the entire source code into the final image, we copy only the generated JAR:

```
COPY --from=build \
    /app/target/demo-0.0.1-SNAPSHOT.jar \
    app.jar
```

The resulting production image therefore contains only what is required to run the application.

14. `.dockerignore`

Create:

```
.dockerignore
```

inside:

```
~/Desktop/spring-mysql-demo
```

Contents:

```
target  
.git  
.idea  
*.iml  
.DS_Store
```

These files and directories do not need to be included in the Docker build context.

15. Build the Docker Image

Move into the project:

```
cd ~/Desktop/spring-mysql-demo
```

Build the image:

```
docker build -t demo:1.0.0 .
```

Breakdown:

```
docker build
  ↓
Build a Docker image

-t demo:1.0.0
  ↓
Image repository/name + tag

.
↓
Current directory is the build context
```

Verify:

```
docker images
```

We now have:

```
Source Code
  ↓
Dockerfile
  ↓
Multi-Stage Build
  ↓
demo:1.0.0
```

16. Why We Cannot Run Only the Spring Image

Technically, we could try:

```
docker run demo:1.0.0
```

But our Spring Boot application depends on MySQL.

Running the complete application manually would require us to manage:

- MySQL container
- Spring container
- Docker network
- Volume
- Ports
- Environment variables
- Startup ordering

This is where **Docker Compose** becomes useful.

17. Docker Compose

Create:

```
compose.yaml
```

inside:

```
~/Desktop/spring-mysql-demo
```

Use:

```
services:

  db:
    image: mysql:8.4.7

    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: appdb
      MYSQL_USER: appuser
      MYSQL_PASSWORD: apppass

  ports:
```

```

    - "3306:3306"

volumes:
  - mysql-data:/var/lib/mysql

networks:
  - spring-net

healthcheck:
  test:
    [
      "CMD-SHELL",
      "mysqladmin ping -h localhost -uroot -p$$MYSQL_ROOT_PASSWORD --silent"
    ]
  interval: 5s
  timeout: 5s
  retries: 10

app:
  image: demo:1.0.0

  environment:
    DB_URL: "jdbc:mysql://db:3306/appdb?useSSL=false&allowPublicKeyRetrieval=true"
    DB_USERNAME: appuser
    DB_PASSWORD: apppass

  ports:
    - "8080:8080"

  depends_on:
    db:
      condition: service_healthy

  networks:
    - spring-net

volumes:
  mysql-data:

networks:
  spring-net:
    driver: bridge

```

18. MySQL Image

```
image: mysql:8.4.7
```

We are not building MySQL ourselves.

If the image does not already exist locally, Docker can retrieve the official MySQL image.

19. MySQL Environment Variables

```
environment:  
  MYSQL_ROOT_PASSWORD: rootpass  
  MYSQL_DATABASE: appdb  
  MYSQL_USER: appuser  
  MYSQL_PASSWORD: apppass
```

MYSQL_ROOT_PASSWORD

```
rootpass
```

Sets the password for MySQL's root user.

The root user has administrative privileges over the MySQL server.

MYSQL_DATABASE

```
appdb
```

Creates the database:

```
appdb
```

during the initial MySQL container setup.

MYSQL_USER

```
appuser
```

Creates a normal database user.

Applications should generally connect using an application-specific user rather than the root account.

MYSQL_PASSWORD

```
apppass
```

Sets the password for:

```
appuser
```

20. Port Mapping

MySQL uses:

```
ports:
  - "3306:3306"
```

The basic format is:

```
HOST_PORT : CONTAINER_PORT
```

Similarly, Spring Boot uses:

```
ports:
  - "8080:8080"
```

Therefore:

```
Mac localhost:8080
  ↓
Spring Container:8080
```

21. Persistent MySQL Storage

The Compose file contains:

```
volumes:
  - mysql-data:/var/lib/mysql
```

Without a volume, database files would primarily live in the container's writable layer.

If the container were removed and recreated, that data could be lost.

With a named volume:

```
MySQL Container
  |
  ▼
/var/lib/mysql
  |
  ▼
Docker Volume
  |
  ▼
mysql-data
```

The volume has a lifecycle separate from the individual MySQL container.

22. Connecting Spring Boot to MySQL

Suppose Spring Boot contains a property similar to:

```
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/appdb}
```

Compose provides:

```
DB_URL: "jdbc:mysql://db:3306/appdb?useSSL=false&allowPublicKeyRetrieval=true"
```

Notice the hostname:

```
db
```

not:

```
localhost
```

Inside the Spring container:

```
localhost
```

would refer to the **Spring container itself**.

MySQL is running in another container.

Because the Compose service is called:

```
db:
```

Spring can connect using:

```
jdbc:mysql://db:3306/appdb
```

23. Docker Network

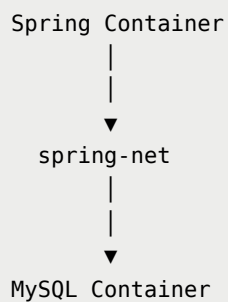
Both services are connected to:

```
networks:
  - spring-net
```

And the network is defined as:

```
networks:
  spring-net:
    driver: bridge
```

Conceptually:



Docker networking allows the two containers to communicate with each other.

24. Why We Need a Healthcheck

Starting a MySQL container does not necessarily mean MySQL is immediately ready for database connections.

There is an important difference between:

```
MySQL container is running
```

and:

```
MySQL is ready to accept JDBC connections
```

MySQL may require some time for initialization.

The healthcheck verifies its readiness:

```
healthcheck:
  test:
    [
      "CMD-SHELL",
      "mysqladmin ping -h localhost -uroot -p$MYSQL_ROOT_PASSWORD --silent"
    ]
  interval: 5s
  timeout: 5s
  retries: 10
```

The application then uses:

```
depends_on:
  db:
    condition: service_healthy
```

The startup flow becomes:

```
Start MySQL Container
  ↓
Wait for MySQL to become healthy
  ↓
Start Spring Boot Container
```

25. Validate the Compose File

Before starting the services:

```
cd ~/Desktop/spring-mysql-demo
```

Run:

```
docker compose config
```

This is useful for catching YAML syntax or indentation problems.

26. Start the Complete Application

Run:

```
docker compose up -d
```

Meaning:

```
docker compose up
      ↓
Create and start the services

-d
  ↓
Detached mode
```

Detached mode keeps the containers running in the background.

Because the Spring application image already exists locally as:

```
demo:1.0.0
```

Compose can use that image directly.

27. View Running Containers

Run:

```
docker compose ps
```

Both services should eventually appear:

```
app  
db
```

You can also use:

```
docker ps
```

28. View Container Logs

MySQL logs:

```
docker compose logs db
```

Spring Boot logs:

```
docker compose logs app
```

Follow Spring logs continuously:

```
docker compose logs -f app
```

Exit log-following mode with:

```
Ctrl + C
```

This stops following the logs. It does not stop the containers.

29. Test the Dockerized Application

Send a request:

```
curl -X POST "http://localhost:8080/notes?text=Running-In-Docker"
```

Expected response:

```
{
  "id": 1,
  "text": "Running-In-Docker"
}
```

Send another:

```
curl -X POST "http://localhost:8080/notes?text=Compose-Works"
```

At this point, the request travels approximately like this:

```
curl
↓
localhost:8080
↓
Spring Boot Container
↓
Docker Network
↓
MySQL Container
↓
Persistent Docker Volume
```

30. Verify the MySQL Data

Run:

```
docker compose exec db \
mysql -uappuser -papppass appdb \
-e "SELECT * FROM notes;"
```

Expected output:

```
+-----+
| id | text |
+-----+
| 1 | Running-In-Docker |
| 2 | Compose-Works |
+-----+
```

The complete local environment now contains:

```
Spring Boot
+
MySQL
+
Docker Networking
+
Persistent Storage
```

31. Inspect the Docker Network

List networks:

```
docker network ls
```

The Compose network name may include the Compose project name.

For example:

```
demo_spring-net
```

Inspect it:

```
docker network inspect demo_spring-net
```

If the generated name is different, use the actual name shown by:

```
docker network ls
```

The inspection output will show both containers attached to the network.

32. Stop the Local Environment

From the project directory:

```
docker compose down
```

This removes:

- Containers
- Compose network

The named MySQL volume remains.

Check:

```
docker volume ls
```

Because the volume still exists, the database data can survive container recreation.

Delete Containers and Database Data

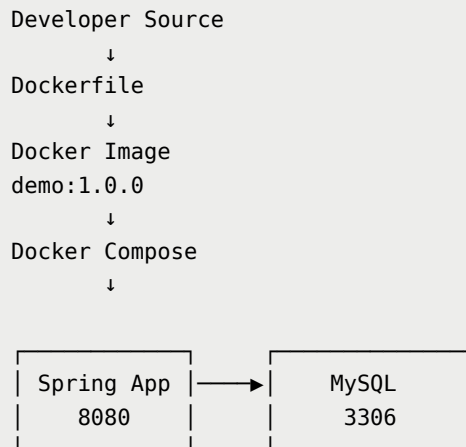
To remove the volume as well:

```
docker compose down -v
```

Do not use `-v` when you want to preserve your database data.

33. What We Have Achieved Locally

Everything up to this point has happened on one developer machine:



There is still one major problem.

The image:

```
demo:1.0.0
```

exists only on the developer's Mac.

An Ubuntu server cannot automatically access an image stored inside the Mac's local Docker image store.

This is where a **container registry** becomes useful.

34. Create a Docker Hub Repository

Create or sign in to a Docker Hub account.

Suppose the username is:

```
codarmy
```

Create a repository named:

```
spring-mysql-demo
```

For a simple classroom demo, it can be made public.

The resulting repository reference becomes:

```
codermay/spring-mysql-demo
```

Replace `codermay` with your own Docker Hub username.

35. Login to Docker Hub

From the Mac terminal:

```
docker login
```

Complete the authentication flow.

36. Tag the Local Image

Our local image is:

```
demo:1.0.0
```

Docker Hub needs a reference containing our namespace:

```
USERNAME/spring-mysql-demo:1.0.0
```

For example:

```
docker tag \  
demo:1.0.0 \  

```

```
codersarmy/spring-mysql-demo:1.0.0
```

Now check:

```
docker images
```

You may see:

```
demo                1.0.0
codersarmy/spring-mysql-demo  1.0.0
```

Both names can point to the same underlying image.

Tagging does not mean that Docker has created another complete copy of the image.

It has created another reference to it.

37. Push the Image to Docker Hub

Run:

```
docker push codersarmy/spring-mysql-demo:1.0.0
```

Docker uploads the required image layers to the registry.

After the push completes, Docker Hub should contain:

```
Repository:
spring-mysql-demo
```

```
Tag:
1.0.0
```

The application image is no longer limited to the developer's machine.

38. Why Don't We Push MySQL?

Our custom Spring Boot application needs to be pushed because we built it ourselves.

MySQL is different.

The image:

```
mysql:8.4.7
```

already exists in a public registry.

Therefore, the Ubuntu server can obtain:

```
codarmy/spring-mysql-demo:1.0.0
```

from our repository and:

```
mysql:8.4.7
```

from the official MySQL repository.

39. What Needs to Go to the Ubuntu Server?

The server does **not** need:

```
src/  
target/  
pom.xml  
mvnw  
Dockerfile  
JAR
```

The application has already been:

```
Source Code
  ↓
Built into Docker Image
  ↓
Stored in Docker Hub
```

The server mainly needs the deployment configuration that tells Docker which images to run.

For this demo, that configuration is the production Compose file.

This is one of the major advantages of the container workflow.

40. Production Compose File

Keep the local and server configuration separate.

Create:

```
compose.prod.yaml
```

Example:

```
services:

  db:
    image: mysql:8.4.7

    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: appdb
      MYSQL_USER: appuser
      MYSQL_PASSWORD: apppass

    ports:
      - "3306:3306"

    volumes:
      - mysql-data:/var/lib/mysql

    networks:
      - spring-net
```

```

healthcheck:
  test:
    [
      "CMD-SHELL",
      "mysqladmin ping -h localhost -uroot -p$$MYSQL_ROOT_PASSWORD --silent"
    ]
  interval: 5s
  timeout: 5s
  retries: 10

restart: unless-stopped

app:
  image: coderarmy/spring-mysql-demo:1.0.0

  environment:
    DB_URL: "jdbc:mysql://db:3306/appdb?useSSL=false&allowPublicKeyRetrieval=true"
    DB_USERNAME: appuser
    DB_PASSWORD: apppass

  ports:
    - "8080:8080"

  depends_on:
    db:
      condition: service_healthy

  networks:
    - spring-net

  restart: unless-stopped

volumes:
  mysql-data:

networks:
  spring-net:
    driver: bridge

```

Replace:

```

coderarmy

```

with your own Docker Hub username.

41. Local vs Production Image Reference

Local Compose uses:

```
image: demo:1.0.0
```

That works because the image exists inside the developer machine's Docker image store.

Production uses:

```
image: coderarmy/spring-mysql-demo:1.0.0
```

Now the image has a registry-compatible reference.

If it is not available locally on the server, Docker can retrieve it from Docker Hub.

42. Restart Policy

The production Compose file adds:

```
restart: unless-stopped
```

for both services.

This instructs Docker to restart the containers when necessary unless they were intentionally stopped.

43. Validate the Production Compose File

From the directory containing the production file:

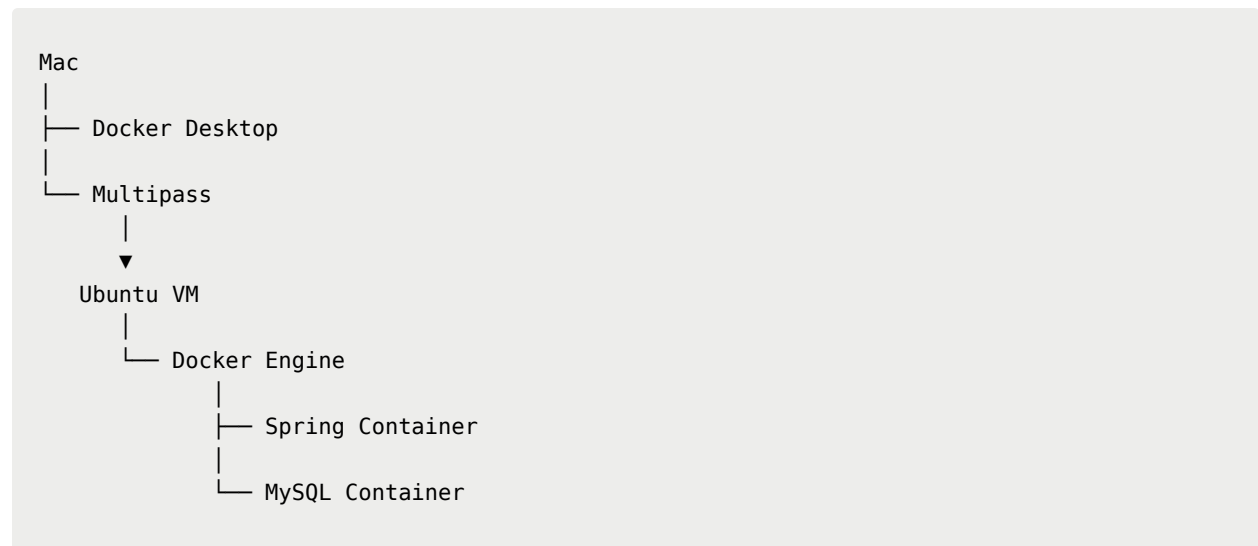
```
docker compose -f compose.prod.yaml config
```

This validates the Compose configuration before using it on the server.

44. Creating the Ubuntu Server with Multipass

Multipass creates an **Ubuntu virtual machine**, not a Docker container.

Our architecture becomes:



This gives us a useful approximation of deploying the application to a separate Linux server.

45. Start Docker on Ubuntu

Inside the Ubuntu VM:

```
sudo systemctl enable --now docker
```

Check the Docker service:

```
sudo systemctl status docker
```

Press:

```
q
```

to exit the status screen if required.

46. Verify Docker on Ubuntu

Check Docker:

```
sudo docker --version
```

Check Docker Compose:

```
sudo docker compose version
```

Run a test container:

```
sudo docker run hello-world
```

If the test succeeds, the Ubuntu machine is ready to run Docker containers.

47. Create the Deployment Directory

Inside Ubuntu:

```
mkdir -p ~/spring-mysql-deploy
```

This directory will hold the deployment configuration used to start the application on the Ubuntu machine.

Complete Flow So Far

The important idea behind the entire demo can be summarized as:



Spring Boot
Container

MySQL
Container

The important shift is that we are no longer thinking in terms of:

"How do I copy my project to the server?"

Instead, the deployment model becomes:

```
Build the application once
    ↓
Package it as an image
    ↓
Publish that image
    ↓
Let the server pull and run the image
```

That is the fundamental container-based deployment workflow.